

B4 輪講 2026：EX7 レポート

加藤 稔之典

2026 年 6 月 28 日

本課題では、OpenSTARLab の preprocessing と rlearn を用いて、FIFA World Cup 2022 のイベントデータ・トラッキングデータから強化学習モデルを作成する。サッカーの状態を SAR 形式に前処理し、観測データを作成したうえで、行動価値を表す Q 値を学習・可視化することを目的とする。

1 課題 1

FIFA World Cup 2022 のオープンデータをダウンロードし、入力データとして使用した。具体的には、match_id が 3812, 3813, 3814, 3815, 3816 のイベントデータ (Event Data)、トラッキングデータ (Tracking Data)、メタデータ (Metadata)、ロスターデータ (Rosters)、および players.csv を取得した。本課題では、その中から 3812, 3813, 3814 の 3 試合を抽出して前処理を行った。

以下のプログラムコード 1 を実行し、SAR 形式の前処理データを作成した。

プログラムコード 1: preprocessing_ffawc2022.py

```
1 from pathlib import Path
2 from preprocessing import SAR_data
3
4 # Step 2: Define your data path as a Path object
5 data_directory = Path("./data/fifawc")
6 config_path = Path("./content/config/preprocessing_ffawc2022.json")
7 # Step 3: Pass the Path object to the class
8 SAR_data(
9     data_provider="fifawc",
10    data_path=data_directory,
11    state_def="PVS",
12    config_path=config_path,
13    preprocess_method="SAR",
14    match_id=3812,
15    max_workers=1
16).preprocess_data()
17
18 SAR_data(
19     data_provider="fifawc",
20     data_path=data_directory,
21     state_def="PVS",
```

```

22     config_path=config_path,
23     preprocess_method="SAR",
24     match_id=3813,
25     max_workers=1
26 ).preprocess_data()
27
28 SAR_data(
29     data_provider="fifawc",
30     data_path=data_directory,
31     state_def="PVS",
32     config_path=config_path,
33     preprocess_method="SAR",
34     match_id=3814,
35     max_workers=1
36 ).preprocess_data()
37
38 print("\n ☑ Preprocessing completed successfully!")

```

1.0.1 SAR 形式に変換する目的

生のサッカーデータ（選手の位置座標の羅列やイベントの記録）のままでは、AI は学習を行うことができない。強化学習は「現在の状態（State）を観測し、行動（Action）を選択し、その結果として報酬（Reward）を得る」というマルコフ決定過程の枠組みに基づいている。そのため、時系列のトラッキングデータとイベントデータを紐づけ、この SAR（State, Action, Reward）の形式にデータを成形し直す必要があった。

次に、以下のプログラムコード 2 を実行し、データを train, validation, test に分割した。

プログラムコード 2: split_data_fifawc2022.py

```

1  from rlearn import RLearn_Model
2  from pathlib import Path
3
4  # デバッグのために、RLearn_Model に渡される前にゲーム ID が検出されるか確認します。
5  input_data_path = Path("./data/fifawc/preprocess_data/")
6
7  game_ids = []
8  if input_data_path.exists():
9      for item in input_data_path.iterdir():
10         if item.is_dir() and item.name.isdigit():
11             game_ids.append(item.name)
12
13  print(f"Detected game IDs: {game_ids}")
14
15  # もしゲーム ID が検出されていなければ、ここで処理を停止できます。
16  if not game_ids:
17      print("エラー: 検出されたゲーム ID がありません。データを分割できません。")
18  else:

```

```

19     RLearn_Model(
20         state_def="PVS",
21         input_path=str(input_data_path),
22         output_path="./data/fifawc/preprocess_data/split/",
23     ).run_rlearn(
24         run_split_train_test=True,
25         run_preprocess_observation=False,
26         run_train_and_test=False,
27         run_visualize_data=False
28     )
29
30     print("\n ☑ Splitting data completed successfully!")

```

以下のプログラムコード 2 を実行し、強化学習で使う observation データを作成した。

1.0.2 データの分割結果

分割後のデータを検証した結果、以下のように試合ごとに完全に独立して割り当てられていることが確認できた。

Train（学習用）： match_id 3814（カタール vs エクアドル） / 710 行

Validation（検証用）： match_id 3813（イングランド vs イラン） / 695 行

Test（テスト用）： match_id 3812（オランダ vs セネガル） / 820 行

各データセットで試合が明確に分かれているため、学習時にテストデータが混ざってしまうデータリーク（Leakage）は発生していない。

続いて、以下のプログラムコード 3 を実行し、強化学習で実際に入力として使う observation データを作成した。

プログラムコード 3: observation_fifawc2022.py

```

1
2     from pathlib import Path
3     from rlearn import RLearn_Model
4     from rlearn.sports.main_class import PreprocessObservationConfig
5
6     config_path = Path("./content/config/preprocessing_fifawc2022.json")
7
8
9     split_data_base_path = "./data/fifawc/preprocess_data/split/"
10
11     RLearn_Model(
12         state_def="PVS",
13         config=config_path,
14         num_process=1,
15         input_path=split_data_base_path + "train",
16         output_path="./data/fifawc/observation_data/split/train",
17     ).run_rlearn(

```

```

18     run_split_train_test=False,
19     run_preprocess_observation=True,
20     run_train_and_test=False,
21     run_visualize_data=False,
22     preprocess_config=PreprocessObservationConfig(batch_size=8)
23 )
24
25 RLearn_Model(
26     state_def="PVS",
27     config=config_path,
28     num_process=1,
29     input_path=split_data_base_path + "validation",
30     output_path="./data/fifawc/observation_data/split/validation",
31 ).run_rlearn(
32     run_split_train_test=False,
33     run_preprocess_observation=True,
34     run_train_and_test=False,
35     run_visualize_data=False,
36     preprocess_config=PreprocessObservationConfig(batch_size=8)
37 )
38
39 RLearn_Model(
40     state_def="PVS",
41     config=config_path,
42     num_process=1,
43     input_path=split_data_base_path + "test",
44     output_path="./data/fifawc/observation_data/split/test",
45 ).run_rlearn(
46     run_split_train_test=False,
47     run_preprocess_observation=True,
48     run_train_and_test=False,
49     run_visualize_data=False,
50     preprocess_config=PreprocessObservationConfig(batch_size=8)
51 )
52
53 print("\n ☑ Creating Observation data completed successfully!")

```

1.0.3 observation のサンプル

作成された Test データ (match.id: 3812) の 1 行目をサンプルとして抽出した。このシーケンスの対象選手 (エゴプレイヤー) は、オランダ代表の Virgil van Dijk (DF) である。状態 (State / observation) : その瞬間のピッチ上の環境情報を表す。データには、ボールの現在座標 ($x: 1.108, y: 0.339$) や速度、ボール保持者 (エゴプレイヤー) の座標と速度、そしてピッチ上にいる味方・相手の全 22 選手の位置座標、速度、役割 (DF, MF など) が辞書型で記録されている。行動 (Action) : この状況下で対象選手が実際に選択したプレーを表す。本サンプルでは up と記録されており、前進 (または上方向へのプレー) を選択したことがわかる。エージェントは、この observation (全選手とボールの空間的・物理的な配置) を入力として受け取ることで、

「味方がどこにいるか、敵のディフェンスラインがどうなっているか」といった現在の状況を把握する。そして、「この状況ならどの行動をとれば最も将来の報酬が高くなるか」という Q 値 を算出するための唯一の情報源として機能する。observation データは、AI にとっての「視覚情報・意思決定の判断材料」であるといえる。

2 課題 2：Q 値の可視化と考察

本課題では、強化学習モデルを用いて学習を行い、特定の試合場面における各行動の Q 値（行動価値）を可視化して考察を行った。

まず、プログラムコード 4 を実行し、学習のための設定ファイル（config）を更新した。

プログラムコード 4: train_fifawc2022.py

```
1 import json
2
3 config_path = "./content/config/exp_fifawc2022.json"
4
5 with open(config_path, 'r') as f:
6     config_data = json.load(f)
7
8 print(config_data)
9
10 # Modify some values in the config_data dictionary
11 config_data['dataset']['train_filename'] = './data/fifawc/observation_data/split/train'
12 config_data['dataset']['valid_filename'] = './data/fifawc/observation_data/split/validation'
13 config_data['dataset']['test_filename'] = './data/fifawc/observation_data/split/test'
14
15 print(f"config: {config_data['dataset']['test_filename']}")
16
17 # changed learning rate and max epoch
18 print(f"max epochs: {config_data['max_epochs']}, learning rate:
19     ↳ {config_data['model']['optimizer']['lr']}")
20
21 # Modify some values in the config_data dictionary
22 config_data['max_epochs'] = 1 # Example: Change max_epochs to 30
23 config_data['model']['optimizer']['lr'] = 0.01 # Example: Change learning rate to 0.001
24 config_data['datamodule']['type'] = 'rl_attacker' # Fix the datamodule type
25
26 # You can modify other values as needed
27 # config_data['datamodule']['batch_size'] = 128
28
29 # Print the modified values to confirm
30 print(f"max epochs: {config_data['max_epochs']}, learning rate:
31     ↳ {config_data['model']['optimizer']['lr']}")
32 print(f"datamodule type: {config_data['datamodule']['type']}")
33
34 with open(config_path, 'w') as f:
35     json.dump(config_data, f, indent=4)
```

```

34
35 print(f"Modified config saved to {config_path}")

```

次に、プログラムコード 5 を実行し、不均衡データに対応するためのクラスウェイト (class weights) を計算した。

プログラムコード 5: train.fifawc2022.class_weight_only.py

```

1  import json
2  from pathlib import Path
3
4  from rlearn import RLearn_Model
5  from rlearn.sports.main_class import TrainAndTestConfig
6
7  base_config_path = Path("./content/config/exp_fifawc2022.json")
8  cached_config_path = Path("./content/config/exp_fifawc2022_with_cached_class_weight.json")
9  class_weight_cache_path =
10     ↳ Path("./content/output/class_weight/exp_fifawc2022_class_weights.pt")
11
12  config = json.loads(base_config_path.read_text(encoding="utf-8"))
13  if "class_weight_fn" not in config:
14      raise ValueError(
15          "`~/content/config/exp_fifawc2022.json` に `class_weight_fn` がありません。"
16          "先に `class_weight_fn.type` などを設定してください。"
17      )
18
19  # Set num_workers to 0 to avoid DataLoader worker issues
20  config["datamodule"]["num_workers"] = 0
21
22  config["class_weight_fn"]["cache_path"] = str(class_weight_cache_path)
23
24  cached_config_path.parent.mkdir(parents=True, exist_ok=True)
25  cached_config_path.write_text(
26      json.dumps(config, ensure_ascii=False, indent=4),
27      encoding="utf-8",
28  )
29
30  RLearn_Model(
31      state_def="PVS"
32  ).run_rlearn(
33      run_split_train_test=False,
34      run_preprocess_observation=False,
35      run_train_and_test=True,
36      run_visualize_data=False,
37      train_and_test_config=TrainAndTestConfig(
38          exp_name="sarsa_attacker",
39          run_name="test",

```

```

39         accelerator="gpu",
40         devices=1,
41         strategy="auto",
42         exp_config_path=str(cached_config_path),
43         use_class_weights=True,
44         output_base_dir="./content/output",
45         class_weight_only=True,
46     ),
47 )
48
49 print(f"\n☑ Class weights saved to: {class_weight_cache_path}")
50 print("☑ Class-weight-only step completed successfully!")

```

続いて、計算したクラスウェイトを用いて、プログラムコード 6 により強化学習モデルの学習を行った。

プログラムコード 6: train.fifawc2022_with_cached_class_weight.py

```

1  import json
2  from pathlib import Path
3
4  from rlearn import RLearn_Model
5  from rlearn.sports.main_class import TrainAndTestConfig
6
7  base_config_path = Path("./content/config/exp_fifawc2022.json")
8  cached_config_path = Path("./content/config/exp_fifawc2022_with_cached_class_weight.json")
9  class_weight_cache_path =
10     ↪ Path("./content/output/class_weight/exp_fifawc2022_class_weights.pt")
11
12 config = json.loads(base_config_path.read_text(encoding="utf-8"))
13 if "class_weight_fn" not in config:
14     raise ValueError(
15         "`/content/config/exp_fifawc2022.json` に `class_weight_fn` がありません。"
16         "先に `class_weight_fn.type` などを設定してください。"
17     )
18
19 # Set num_workers to 0 to avoid DataLoader worker issues
20 config["datamodule"]["num_workers"] = 0
21
22 config["class_weight_fn"]["cache_path"] = str(class_weight_cache_path)
23
24 cached_config_path.parent.mkdir(parents=True, exist_ok=True)
25 cached_config_path.write_text(
26     json.dumps(config, ensure_ascii=False, indent=4),
27     encoding="utf-8",
28 )
29
30 if not class_weight_cache_path.exists():
31     raise FileNotFoundError(

```

```

31         f"Cached class weights not found: {class_weight_cache_path}\n"
32         "先に `train_fifawc2022_class_weight_only.py` を実行してください。"
33     )
34
35     RLern_Model(
36         state_def="PVS"
37     ).run_rlearn(
38         run_split_train_test=False,
39         run_preprocess_observation=False,
40         run_train_and_test=True,
41         run_visualize_data=False,
42         train_and_test_config=TrainAndTestConfig(
43             exp_name="sarsa_attacker",
44             run_name="test",
45             accelerator="gpu",
46             devices=1,
47             strategy="auto",
48             exp_config_path=str(cached_config_path),
49             use_class_weights=True,
50             output_base_dir="./content/output",
51             class_weight_only=False,
52             save_q_values_csv=True,
53             max_games_csv=1,
54             max_sequences_per_game_csv=5,
55         ),
56     )
57
58     print("\n ☑ Training completed successfully with cached class weights!")

```

学習完了後、プログラムコード 7 を実行し、学習済みモデルに基づく Q 値を画像として可視化した。設定として `match\_id="3814"`、`sequence\_id=0`、`viz\_style="bar"`、`keep\_frames=True` を指定している。

プログラムコード 7: visualize_qvalues.py

```

1  from pathlib import Path
2  import os
3  from rlearn import RLern_Model
4  from rlearn.sports.main_class import VisualizeDataConfig
5
6  config_path = Path("./content/config/exp_fifawc2022.json")
7
8  RLern_Model(
9      state_def="PVS"
10 ).run_rlearn(
11     run_split_train_test=False,
12     run_preprocess_observation=False,
13     run_train_and_test=False,

```



```

14     run_visualize_data=True,
15     visualize_config=VisualizeDataConfig(
16         exp_config_path=config_path,
17
18         ↪ checkpoint_path="./content/output_epoch5/sarsa_attacker/test/checkpoints/best-04-30-val_loss=0.07",
19         model_name="exp_fifawc2022_with_cached_class_weight",
20         tracking_file_path="./data/fifawc/preprocess_data/3814/events.jsonl",
21         match_id="3814",
22         sequence_id=0,
23         viz_style="bar",
24         movie_output_dir="./content/output_epoch5",
25         keep_frames=True
26     ),
27
28     print("\n ⚠ Updated visualize_qvalues.py without unsupported argument!")

```

2.1 Q 値の可視化結果と考察

抽出したフレーム 79、254、363 の 3 つの場面（match_id: 3814、対象選手: Pedro Miguel）における Q 値の可視化結果を図 1～3 に示す。各場面について、AI が高く評価した行動と実際のピッチ上の状況との整合性を考察する。

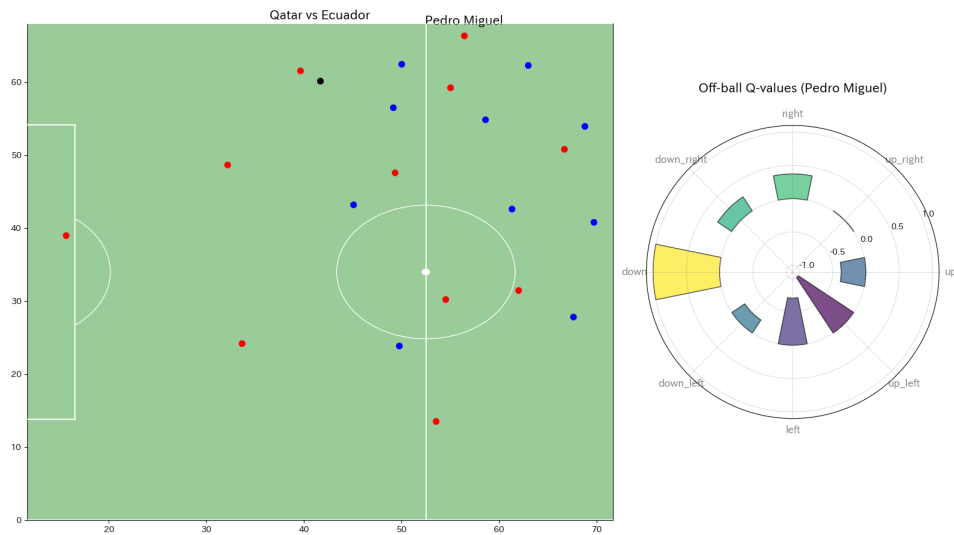


図 1: フレーム 0079 における可視化された Q 値

【場面 1：フレーム 0079 の考察】 図 1 において、最も Q 値が高い行動は「down」（ピッチ図における左方向、すなわち自陣方向への移動）であり、2 番目に高い行動は「right」（ピッチ図の下方向への移動）であった。1 位と 2 位の Q 値の差は大きく開いている。この場面は、対象選手（Pedro Miguel）の付近にいた味方選手が自陣に向かって大きくバックパスを出し、ボールがピッチ左側（後方）へ展開されている状況である。

ここで「down」の価値が高く評価された理由は、ロングパスを受けた味方に対するサポート（パスコースの提供）や、相手へのボールロスト時に備えた守備への切り替えを AI が重視したためだと考えられる。自陣方向へ下がることで、ボール保持の安定性を高めつつ攻撃の組み立て直しを図るという、戦術的に自然な判断となっている。

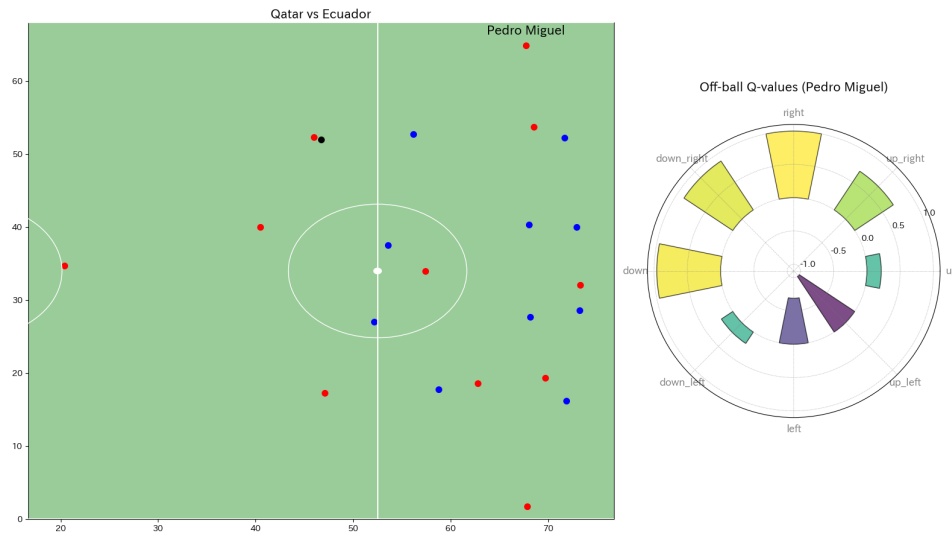


図 2: フレーム 0254 における可視化された Q 値

【場面 2：フレーム 0254 の考察】図 2 において、最も Q 値が高い行動は「down」および「right」であり、次いで「down_right」（左斜め下方向）が高く評価されている。ここでは上位の行動間で Q 値の差が小さいことが特徴である。この差が小さい要因として、現在ボールを保持している味方選手の前方に相手ディフェンダーが 1 名しかおらず、Pedro Miguel 選手から見て自陣方向（down）や右方向（right）にパスを引き出せる広大なスペースが存在していることが挙げられる。どの方向へ動いても味方に有効なパスコースを提供できる状態にあるため、複数のサポート行動が同等に高く評価されている。味方の孤立を防ぎ、ポゼッションを維持して次なる攻撃へ繋げるための適切な評価が行われていると言える。

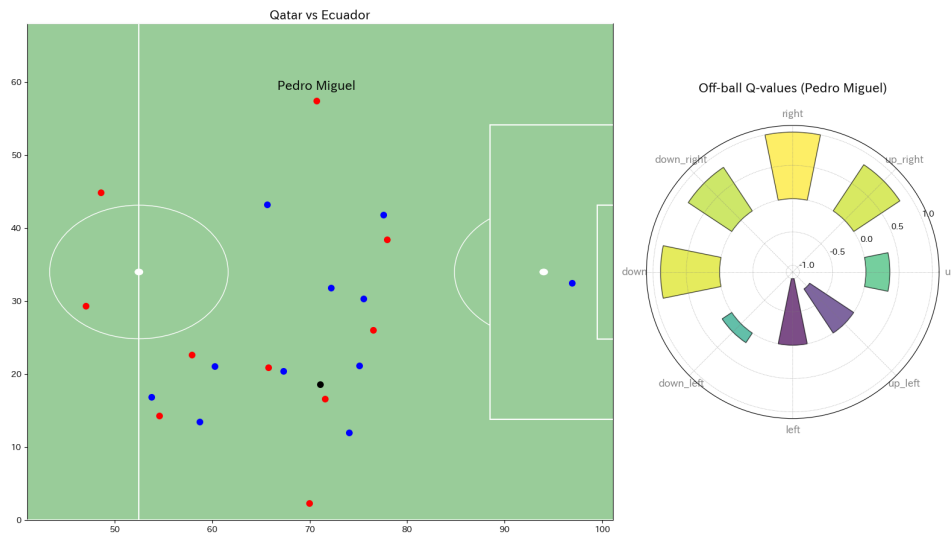


図 3: フレーム 0363 における可視化された Q 値

【場面 3：フレーム 0363 の考察】図 3 において、最も Q 値が高い行動は「right」、次いで「down」となった。この場面では、ボール周辺の味方選手がパスを受け、前線へ攻撃を仕掛けようとしている状況である。ボールホルダーおよびパスを引き出せるスペースが共に「right」方向に存在するため、攻撃のサポートとして「right」が最も高く評価されたと考えられる。また、2 番目に「down」が評価されている理由としては、前線への展開がブロックされた際の「逃げ道（バックパスの選択肢）」を提供するセーフティなサポート行動の価値を AI が学習しているためと推測される。一方で、敵陣深く（up 方向）にも大きなスペースがあるにもかかわらず「up」の Q 値が低く抑えられているのは、相手のディフェンスラインの高さからオフサイド判定を受けるリスクを AI が加味しており、単にスペースへ飛び込むのではなく、ルール上の制約や戦術的なリスクを考慮した高度な判断を下している結果だと考えられる。

3 課題 3：学習設定の変更に伴う Q 値の比較検証

本課題では、ベースラインの学習設定から特定のパラメータを変更して再学習を行い、Q 値の評価や行動の選択にどのような変化が生じるかを検証した。

3.1 実験設定の変更点

設定の変更は以下の 2 パターンで行った。

1. **エポック数の増加（1 から 5 へ）**：モデルの学習回数を増やすことで、Q 値の評価がどう洗練されるかを確認するため、プログラムコード 4 (train_fifawc2022.py) 内の `config.data['max_epochs']` を 1 から 5 へ変更した。
2. **Class weights（クラスの重み付け）の無効化**：データセット内の行動頻度の偏り（不均衡データ）に対する補正効果を確認するため、エポック数 5 の設定のまま、プログラムコード 6 の `use_class_weights=True` を `False` に変更し、重み付けなしで学習を行った。

3.2 エポック数の変更（1 → 5）による影響

エポック数 5 に変更して学習を行った際の、抽出したフレーム 79、254、363 における Q 値の可視化結果を図 4～6 に示す。

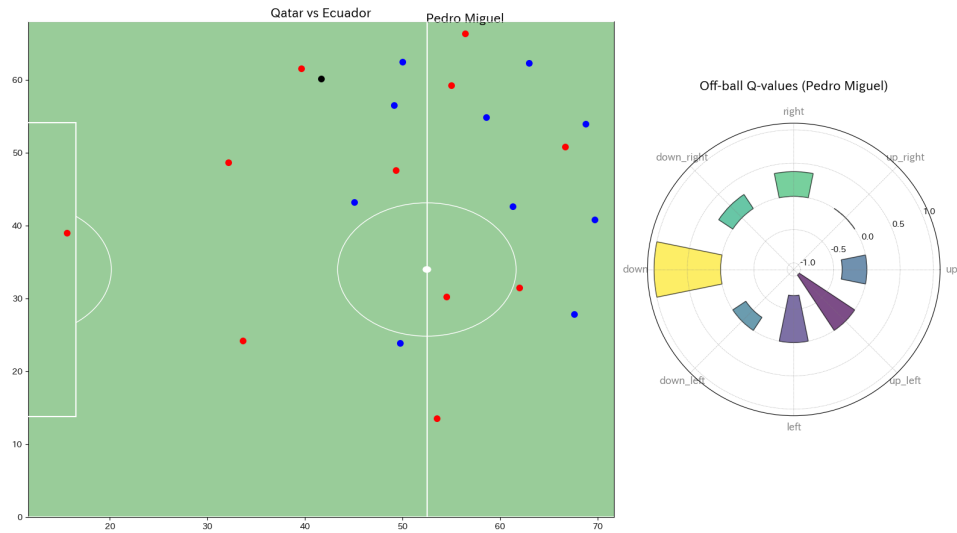


図 4: エポック数 5 で学習を行った際のフレーム 0079 における可視化された Q 値

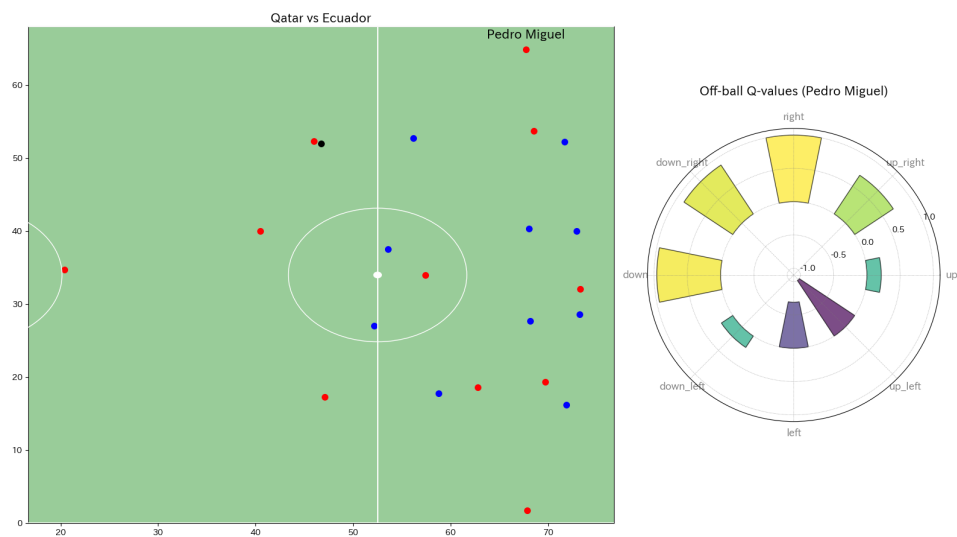


図 5: エポック数 5 で学習を行った際のフレーム 0254 における可視化された Q 値

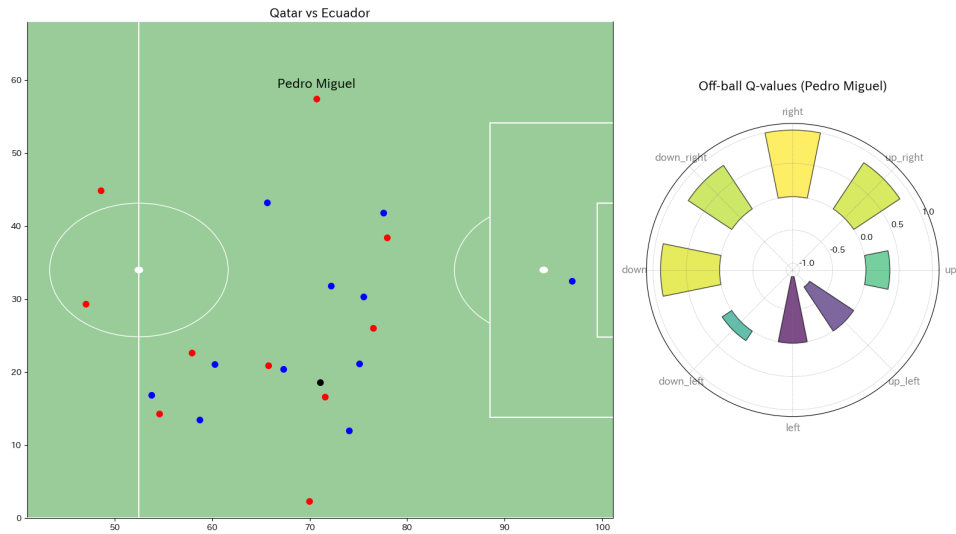


図 6: エポック数 5 で学習を行った際のフレーム 0363 における可視化された Q 値

結果として、エポック数 1（課題 2）の時の結果と見比べても、どの場面においても行動に対する Q 値の分布はほとんど変化しなかった。これは、エポック数 1 の段階ですでに現在の学習率やネットワーク構造における局所最適解にある程度到達しており、数エポックの追加学習では Q 値の相対的な評価を覆すほどの大きな重み更新が発生しなかったためと考えられる。

3.3 Class weights の有無による影響

次に、class weights なし（`use_class_weights=False`）で学習を行った際の Q 値の可視化結果を図 7～9 に示す。

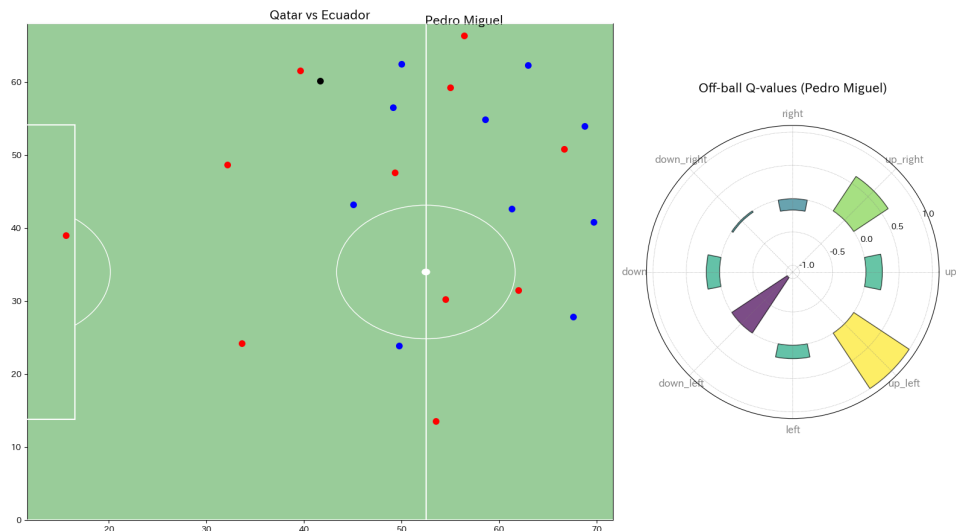


図 7: class weights なしで学習を行った際のフレーム 0079 における可視化された Q 値

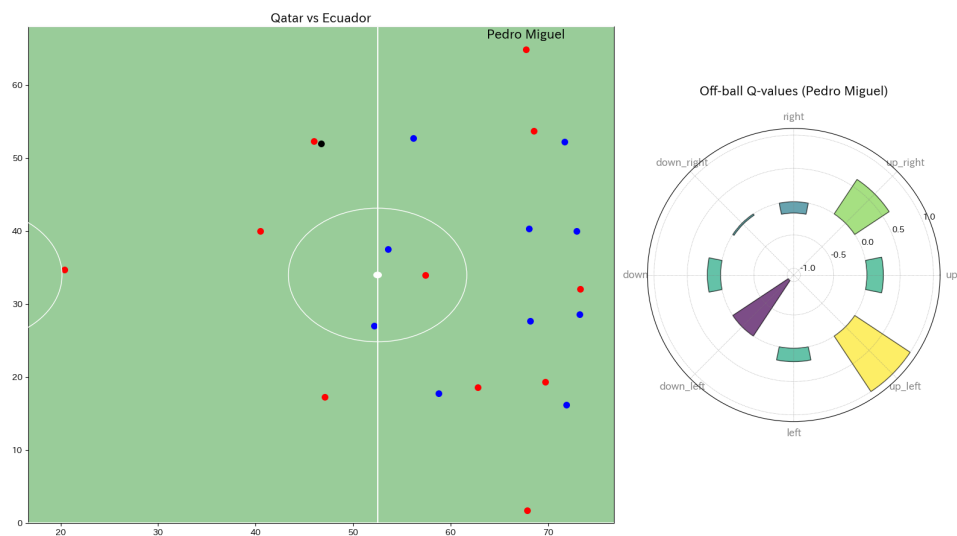


図 8: class weights なしで学習を行った際のフレーム 0254 における可視化された Q 値

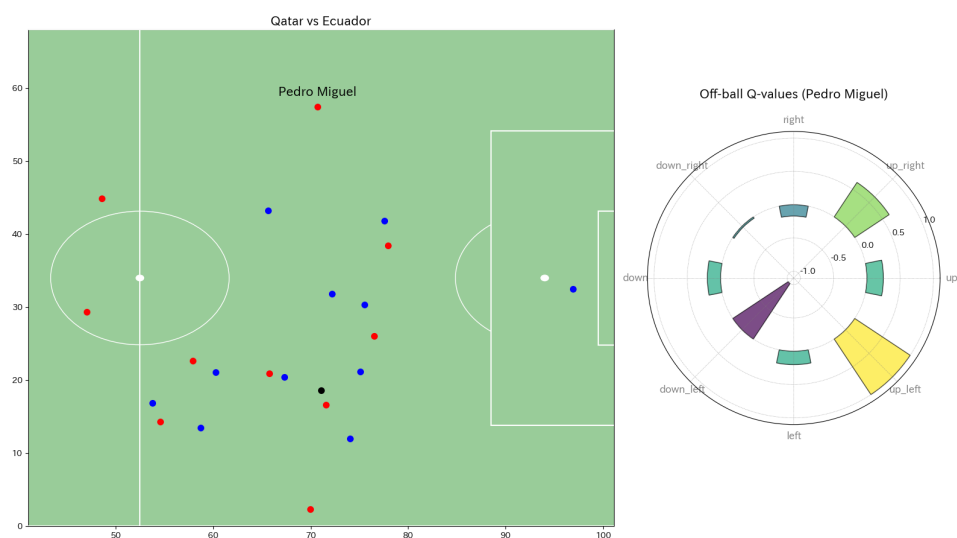


図 9: class weights なしで学習を行った際のフレーム 0363 における可視化された Q 値

図から明らかなように、class weights を無効にした場合、試合の状況や選手の配置が変化しているにもかかわらず、どのフレームでも常に「up_left」の Q 値が最も高く、次いで「up_right」が高くなるという、固定化された全く同じ予測値が出力され続ける結果となった。

3.4 Q 値ランキングの比較と考察

同じ場面（フレーム 79, 254, 363）に対する、3 つの設定間での Q 値ランキングの比較を表 1 に示す。

表 1: 各設定における行動の Q 値ランキング比較

フレーム	エポック数 1 (重みあり)	エポック数 5 (重みあり)	Class weights なし
0079	down > right > down_right > up_right > down_left > up > left > up_left	同左	up_left > up_right > (up, left, down) > down_right > right > down_left
0254	(down, right) > down_right > up_right > (down_left, up) > left > up_left	同左	同上
0363	(down, right) > (down_right, up_right) > up > down_left > up_left > left	同左	同上

表の比較からも、class weights を用いない場合は状況への適応が行われていないことがわかる。この現象は、機械学習におけるデータの不均衡によるものであると考えられる。サッカーのプレーデータにおいて特定の行動（前方へのプレーなど）が元々多く含まれていた場合、AI は単にデータセット内で出現頻度が高い行動を常に選択すれば全体の誤差を小さくできると学習してしまう。class weights を使用することで、頻度の少ない行動に対しても適切な評価が与えられ、状況（ボール位置、スペース、相手の配置）に応じた戦術的な判断が可能になっていたことがわかった。